# 1 Introduction et documentation

Chaque année, en octobre, a lieu à Baisieux, commune du Nord frontalière de la Belgique, la *course du chicon* sur une distance de 15 km.

Il s'agit de gérer le traitement informatique des données : gestion des inscriptions, récolte des performances des concurrents et publication des résultats.

## 1.1 Archive

L'archive fournie contient les 2 dossiers suivants.

- `src/` contient le module `src/Competitor.py`
- `data/` contient deux jeux de données dans quatre fichiers de données : un petit jeu de données pour faire des tests simples (`data/small_inscrits.csv` et `data/small_performances.csv`), et un gros jeu de données (`data/inscrits.csv` et `data/performances.csv`).

## 1.2 module Competitor

Le module `Competitor.py` permet de manipuler des valeurs représentant les compétiteurs de la course. On peut considérer qu'il permet la définition d'un type `Competitor`. L'étude de la documentation permet de déduire les différentes informations contenues dans une telle donnée.

Il est possible d'examiner le code, mais se limiter à la lecture de la documentation (section 1.3) est un bon exercice, car il implique de se limiter à manipuler les données de type `Competitor` via l'**interface** fournie, sans se préoccuper de l'implémentation qui en a été réalisée.

On va constater que le module met à disposition :

- un **constructeur** (`create`)
- des **accesseurs** (`get_XXX`) qui permettent d'obtenir les informations qui décrivent une donnée `Competitor` (en anglais, *bib* vaut dire *dossard*)
- un unique **modificateur** qui concerne la performance d'un compétiteur (`set_performance`).

Remarque : Les manipulations des données de "type" `Competitor` devront se faire à l'aide des fonctions fournies par les modules.

## 1.3 Interface du module Competitor

On définit ici l'interface du module `Competitor`, c'est-à-dire l'ensemble des fonctionnalités qu'il offre.

L'auteur de ce module est le département Informatique de l'Université de Lille (juin 2019).

Chacune de ces fonctions sera décrite par une spécification précisant :

- son nom
- ses paramètres
- la valeur qu'elle renvoie ou l'effet qu'elle produit
- les contraintes d'utilisation (UC pour **Usage Constraint**)

La documentation est en anglais.

`Competitor.create(first_name, last_name, sex, birth_date, bib_num)`

Parameters

- `first_name` – first name of a competitor
- `last_name` – last name of a competitor
- `sex` – sex of a competitor, `"M"` or `"F"`
- `birth_date` (string) – birth date of the competitor, format is `"DD/MM/YYYY"`
- `bib_num` (int) – bib number of the competitor

Returns : a new record for this competitor
Return type : Competitor
UC : `bib_num > 0 and sex in 'MF'`

`Competitor.get_firstname(comp)`

Parameter : `comp` (`Competitor`)
Returns : first name of competitor `comp`
Return type : `str`

`Competitor.get_lastname(comp)`

Parameter : `comp` (`Competitor`)
Returns : last name of competitor `comp`
Return type : `str`

`Competitor.get_birthdate(comp)`

Parameter : `comp` (`Competitor`)
Returns : birthdate of competitor `comp`
Return type : `str`

`Competitor.get_bib_num(comp)`

Parameter : `comp` (`Competitor`)
Returns : bib number of competitor `comp`
Return type `str`

`Competitor.get_performance(comp)`

Parameter : `comp` (`Competitor`)
Returns : performance of competitor `comp`
Return type : `time`

`Competitor.get_sex(comp)`

Parameter : `comp` (`Competitor`)
Returns : sex of competitor `comp`
Return type : `str`

`Competitor.set_performance(comp, d)`

Parameters

- `comp` (`Competitor`) – competitor to be modified
- `d` (`time`) – performance of competitor `comp`

Side effect : performance of competitor `comp` is modified with value `d`

`Competitor.to_string(competitor)`

Parameter : `comp` (`Competitor`)
Returns : a string representation for given competitor, form is
`"[bib] : first last (sex - birth)"`

# 2 Création du module Time

Les performances des compétiteurs vont être représentées par leur temps de course exprimé en heures, minutes et secondes.

Il faut donc définir un module qui offre les fonctionnalités pour gérer des temps : les créer et les manipuler.

A faire : Créer un module `Time.py` qui définit :

- une fonction `create` à trois arguments représentant un nombre d'heures, de minutes et de secondes et qui permet de créer une donnée représentant un temps (on peut envisager un contrôle de validité des valeurs des paramètres).

  Le choix de la structure de données est libre : un dictionnaire (comme pour `Competitor`), un tuple ou une liste ;
- des fonctions donnant accès aux différentes informations d'une donnée de type `temps` ;
- une fonction `compare` qui définit une relation d'ordre sur les données représentant un temps. De manière conventionnelle, le résultat de cette fonction, à deux paramètres représentant des temps, est négatif si son premier paramètre est inférieur au second, positif s'il lui est plus grand et nul quand ils sont égaux.

  NB : il peut être utile de définir une fonction `to_seconds` qui fournit en résultat le nombre de secondes correspondant au temps passé en paramètre.
- une fonction `to_string` qui a pour résultat une représentation sous la forme d'une chaîne de caractères de son paramètre représentant un temps.

La remarque faite à propos du module `Competitor` est valable à nouveau pour l'utilisation du module `Time` : le code ne doit pas dépendre de l'implémentation choisie.

# 3 Gestion des inscrits

Les fonctions suivantes sont à définir dans un module `course_chicon` qui regroupera les fonctions utiles à la gestion de la course.

Il sera bien sûr nécessaire d'importer les modules `Competitor` et `Time`.

Votre première tâche est de récupérer les informations sur les compétiteurs inscrits à la compétition.

Les données concernant ces compétiteurs se trouvent dans le fichier `data/inscrits.csv` (ou `data/small_inscrits.csv`) qui est un fichier au format CSV, c'est-à-dire un fichier texte contenant des données tabulées.

La première ligne de ce fichier est constituée des libellés des données qui suivent :
`Prénoms; Noms; Sexes; Date naiss`

Elle précise donc que chacune des lignes qui suivent contient dans cet ordre le prénom, le nom, le sexe et la date de naissance du compétiteur inscrit. Ces informations sont séparées par un point-virgule.

Avec ces données, on va construire des compétiteurs à l'aide de la fonction `Competitor.create`. Il vous faudra attribuer à chacun de ces compétiteurs un numéro de dossard, obtenu par simple incrémentation d'un compteur, le premier compétiteur recevant le dossard 1.

Tous les compétiteurs seront rassemblés dans un dictionnaire dont les clés seront les numéros de dossard et les valeurs, les compétiteurs associés.

A faire : Réaliser une fonction nommée `read_competitors` paramétrée par le nom du fichier CSV contenant les données des inscrits, qui renvoie le dictionnaire de ces inscrits.

Par exemple, avec le fichier `data/small_inscrits.csv`, la fonction doit retourner :

```
{1: {'bib_num': 1, 'first_name': 'Sidney', 'last_name': 'Robert', 'sex': 'M',
'birth_date': '21/7/1970', 'performance': None},
2: {'bib_num': 2, 'first_name': 'Paien', 'last_name': 'Gilbert', 'sex': 'M',
'birth_date': '26/11/1953', 'performance': None}, ...,
10: {'bib_num': 10, 'first_name': 'Sidney', 'last_name': 'Charest', 'sex': 'M',
'birth_date': '5/3/1981', 'performance': None}}
```

On peut aussi envisager de gérer la situation où aucun fichier ne correspond au paramètre fourni. Cela peut être fait en capturant l'exception `FileNotFoundError` qui est alors déclenchée.

Indication : Penser à la méthode `split` des chaînes de caractères. La méthode `rstrip` peut également être utilisée pour supprimer les marqueurs de fin de ligne.

# 4 Manipulations du dictionnaire

Remarque préliminaire : Le constructeur de listes `list(iterable)` permet de créer une liste dont les éléments sont les valeurs désignées par l'itérable en paramètre. La méthode `values()` des dictionnaires produit un itérable. On peut donc obtenir la liste des valeurs d'un dictionnaire `dic` à l'aide de l'expression `list(dic.values())`.

## 4.1 Affichage

A faire : Ecrire une fonction `show_competitors` qui prend en paramètre une liste de données de type `Competitor` et affiche sur la sortie standard chacune de ces données à raison d'une par ligne (utiliser la fonction `to_string` de `Competitor`).

Utiliser votre fonction pour afficher les compétiteurs contenus dans le dictionnaire produit par la fonction `read_competitors`.

Par exemple, avec le fichier `data/small_inscrits.csv`, la fonction doit retourner :

```
[1]: Sidney Robert (M - 21/07/1970)
[2]: Paien Gilbert (M - 26/11/1953)
[3]: Vincent Riquier (M - 16/09/1980)
[4]: Saville Marier (M - 19/11/1969)
[5]: Namo Lereau (M - 26/03/1980)
[6]: Romaine Hughes (F - 17/10/1943)
[7]: Archard Rivard (M - 10/06/1950)
[8]: Cheney Chassé (M - 21/03/1949)
[9]: Avelaine CinqMars (F - 14/02/1983)
[10]: Sidney Charest (M - 05/03/1981)
```

## 4.2 Sélections

Nous allons écrire quelques fonctions de recherche dans un dictionnaire de valeurs qui satisfont un critère.

Dans cette section, les compétiteurs sont passés en paramètre de chacune des fonctions sous la forme d'un dictionnaire tel que celui construit par la fonction `read_competitors`. Les fonctions à écrire disposent d'un autre paramètre qui correspond, d'une manière ou d'une autre, au critère de sélection des compétiteurs dans le dictionnaire. Les fonctions ont pour résultat soit une donnée de type `Competitor`, soit une liste de telles données. Ce résultat correspond à la sélection selon le critère cherché.

A faire : Ecrire une fonction `select_competitor_by_birth_year` dont le résultat est la liste des compétiteurs dont l'année de naissance correspond à une valeur passée en paramètre.

Suggestion : Etudier la documentation de la fonction `endswith` des chaînes de caractères.

Quel résultat renvoyer si aucun compétiteur ne correspond à l'année fournie ?
NB : Dans le petit jeu de données, deux compétiteurs sont nés en 1980.

A faire : Ecrire une fonction `select_competitor_by_name` dont le résultat est la liste des compétiteurs dont le nom (`last_name`) contient la chaîne de caractères passée en paramètre.

# 5 Report des performances

La course achevée, votre tâche consiste à reporter les performances des compétiteurs dans les fiches de la liste de ces compétiteurs.

## 5.1 Lecture des performances

Les données concernant les performances se trouvent dans le fichier `data/performances.csv` (ou `data/small_performances.csv`) qui est un fichier au format CSV.

La première ligne de ce fichier est constituée des libellés des données qui suivent :
`bib_num;hours;minutes;seconds`

Elle précise donc que chacune des lignes qui suivent contient dans cet ordre le numéro de dossard, le nombre d'heures, de minutes et de secondes du temps de parcours d'un compétiteur, ces informations étant séparées par un point-virgule.

Seuls les compétiteurs ayant effectivement participé et achevé la course figurent dans ce fichier. Avec ces données, on va construire un dictionnaire des performances qui associe à un numéro de dossard une donnée représentant un temps comme définie par le module `Time.py`.

A faire : Ecrire une fonction nommée `read_performances` paramétrée par le nom du fichier CSV contenant les données des performances, qui renvoie le dictionnaire des performances contenues dans ce fichier.

A faire : Tester la validité de votre fonction avec le fichier `data/small_performances.csv`.
Vérifier en particulier la taille du dictionnaire obtenu, ainsi que le contenu de quelques éléments.

## 5.2 Report

Maintenant que l'on dispose des données sur les compétiteurs et leurs performances sous forme de dictionnaires qui partagent les mêmes clés (les numéros de dossard), votre travail consiste à reporter les performances dans les fiches de ces compétiteurs.

A faire : Ecrire une fonction nommée `merge_performances` paramétrée par les deux dictionnaires (celui des compétiteurs et celui des performances) qui modifie les données représentant les compétiteurs en reportant leur performance. Cette fonction ne renvoie pas de valeur.

NB : ne pas oublier pas que le module `Competitor` offre une fonction `set_performance`.

Optionnel : La situation dans laquelle l'un des dossards présents dans le dictionnaire des performances ne correspond pas à un dossard du dictionnaire des compétiteurs pose problème. Dans un tel cas, il est certainement approprié de faire déclencher une exception à la fonction `merge_performances`. On peut créer sa propre exception (`UnknownBibError` par exemple) ou utiliser la classe prédéfinie `ValueError` qui peut convenir ici.

A faire : Tester la validité de votre fonction avec les listes produites par le petit jeu de données.

# 6 Tris

Il est conseillé de revoir le principe du tri par insertion (ou du tri par sélection).

A faire : Compléter le module `Competitor` pour lui ajouter une fonction nommée `compare_lastname` qui définit une relation d'ordre sur les compétiteurs selon l'ordre alphabétique de leurs noms. Plus précisément, `compare_lastname(comp1, comp2)` renvoie `True` si le nom de `comp1` est après le nom de `comp2`, dans l'ordre alphabétique.

Utiliser cette fonction pour définir une fonction `sort_competitors_by_lastname` qui prend en paramètre un dictionnaire de compétiteurs, comme défini précédemment, et a pour résultat la liste des compétiteurs triée par ordre alphabétique de leurs noms.

A faire : Définir une fonction `sort_competitors_by_performance` qui produit la liste des compétiteurs triée par ordre croissant des performances réalisées. Les compétiteurs sans résultat sont placés en fin de liste par ordre alphabétique.

# 7 Publication et sauvegarde des résultats

## 7.1 Affichage des résultats

Il est temps de procéder à la publication des résultats.

A faire : Produire l'affichage des résultats par ordre alphabétique, en précisant le prénom, nom, sexe, numéro de dossard et performance de chaque compétiteur. La fonction sera nommée `print_result_by_last_name` et prendra comme paramètre un dictionnaire de compétiteurs.

A faire : Ecrire une fonction nommée `print_result_by_performance` avec comme paramètre un dictionnaire de compétiteurs, qui affiche les résultats par ordre des performances. Les noms des participants sans performance seront triés par ordre alphabétique.

Par exemple, avec le petit jeu de données, et en supposant que le report des performances a été effectué et la liste de compétiteurs triée par ordre de performance, on pourrait obtenir un affichage de la forme :

```
[7]: Archard Rivard (M - 10/6/1950)      =>  0h46mn31s
[8]: Cheney Chassé (M - 21/3/1949)       =>  0h48mn10s
[4]: Saville Marier (M - 19/11/1969)     =>  0h56mn29s
[5]: Namo Lereau (M - 26/3/1980)         =>  1h 6mn20s
[10]: Sidney Charest (M - 5/3/1981)      =>  1h 6mn38s
[1]: Sidney Robert (M - 21/7/1970)       =>  1h 8mn55s
[6]: Romaine Hughes (F - 17/10/1943)     =>  1h17mn 8s
[3]: Vincent Riquier (M - 16/9/1980)     =>  1h21mn23s
[9]: Avelaine CinqMars (F - 14/2/1983)
[2]: Paien Gilbert (M - 26/11/1953)
```

## 7.2 Sauvegarde des résultats

Enfin pour la pérennité de ces résultats, il est important de les sauvegarder dans un fichier.

A faire : Ecrire une fonction nommée `save_results` paramétrée par un dictionnaire de compétiteurs et un nom de fichier de sauvegarde, qui crée un fichier au format CSV contenant :

- les libellés en première ligne `Num_dossard; Prénom; Nom; Performance`
- les résultats sur les lignes suivantes

```
7;Archard;Rivard; 0h46mn31s
8;Cheney;Chassé; 0h48mn10s
...
```

A faire : Tester votre fonction avec le petit jeu de données puis sauvegarder les résultats complets de la course.

**L'équipe organisatrice de la course du chicon vous remercie chaleureusement pour votre contribution à son bon déroulement.**

# 8 Compléments

## 8.1 Pour les fonctions de sélection

Allons un peu plus loin (optionnel)

On peut constater que les deux fonctions de sélection réalisées sont assez similaires et on pourrait imaginer d'autres fonctions de sélection (par sexe, par tranche d'âge, etc.) qui le seraient tout autant. À chaque fois, il s'agit de filtrer parmi les valeurs du dictionnaire, celles qui satisfont un critère de sélection. Ce critère pourrait être défini par un **prédicat**, c'est-à-dire une fonction dont le résultat est un booléen, dont le paramètre serait un compétiteur. Le résultat de cette fonction est `True` si le compétiteur doit être sélectionné (on dit qu'il vérifie le prédicat) et `False` dans le cas contraire.

A faire : En sachant qu'une fonction peut être le paramètre d'une fonction, définir une fonction `select_competitor` dont le premier paramètre est un dictionnaire de compétiteurs et le second est une fonction prédicat.

Le résultat de `select_competitor` est la liste des compétiteurs qui vérifient le prédicat.

A faire : Définir un prédicat qui vérifie si son paramètre de type `Competitor` est de sexe féminin, puis sans définir de nouvelle fonction, produire la liste des compétiteurs de sexe féminin.

Définir un prédicat qui vérifie si son paramètre de type `Competitor` est né en 1980. Produire la liste des compétiteurs nés en 1980.

Encore un peu plus loin ?

Python permet de définir des fonctions anonymes (comme en javascript). On parle de **lambda** ou **lambda expression** ou **lambda fonction**.

Il s'agit, en Python, de fonctions dont le corps est constitué d'une seule expression et pour lesquelles le return est implicite.

Voici un exemple de lambda qui calcule le carré de son paramètre

```
>>> lambda x : x*x
<function <lambda> at 0x000002B1CA9D9730>
```

On peut s'en servir pour définir une fonction

```
>>> carre = lambda x: x*x
>>> carre(4)
16
```

Il est possible d'avoir plusieurs paramètres :

```
>>> add = lambda x,y: x+y
>>> add(3,6)
9
```

Fondamentalement, les lambdas n'apportent rien de particulier par rapport aux fonctions définies par `def`. Leur intérêt réside principalement dans les situations où l'on souhaite passer une fonction comme paramètre sans avoir à définir une fonction nommée.

Il est possible d'utiliser les lambdas et la fonction `filter` pour écrire une troisième version des fonctions de sélection précédentes qui utilise `filter` qui est de fait une fonction prédéfinie qui réalise le travail demandé. Plus d'infos sur la page
`https://book.pythontips.com/en/latest/map_filter.html`

A faire : Sans utiliser le prédicat qui vérifie qu'un compétiteur est de sexe féminin, mais en utilisant une lambda expression, produire la liste des compétiteurs féminines.

De même, sans utiliser la fonction qui vérifie qu'un compétiteur est né en 1980, mais en utilisant une lambda expression, produire la liste des compétiteurs nés en 1980.

Proposer enfin une seconde version de la fonction `select_competitor_by_birth_year`.

## 8.2 Pour les tris

Allons un peu plus loin

Selon le même principe que pour la sélection, on peut constater des similitudes dans les deux fonctions de tris (`sort_competitors_by_lastname` et `sort_competitors_by_performance`) étudiées dans le sujet. En déduire la définition d'une fonction qui prend en paramètre le dictionnaire et la fonction de comparaison pour produire la liste de compétiteurs triée selon la relation d'ordre.

Et toujours pour aller un peu plus loin

Il est possible d'utiliser la fonction prédéfinie `list.sort` (cf. doc python
`https://docs.python.org/fr/3.7/library/stdtypes.html#list.sort`).
Il est intéressant de consulter au préalable le "Guide pour le tri"
(`https://docs.python.org/fr/3.7/howto/sorting.html`).
C'est l'occasion de rencontrer à nouveau une situation où les lambdas peuvent être utilisées.